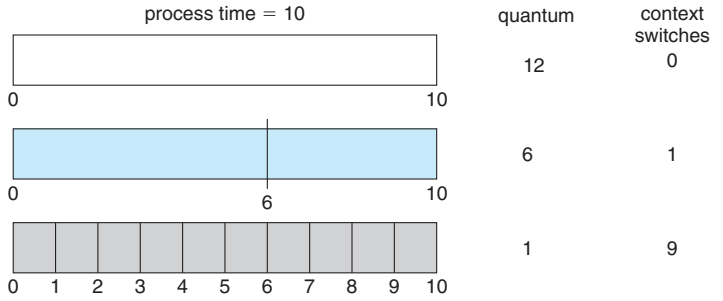


شکل ۵-۵ چگونگی افزایش تعداد برگردان متن با کوچک‌ترشدن کمیت زمانی.

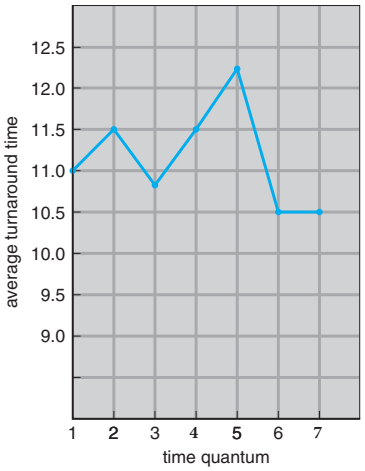
بنابراین، می‌خواهیم کمیت زمانی در مقایسه با زمان برگردان متن تا حدودی بزرگ باشد. اگر زمان برگردان متن، معادل ۱۰ درصد کمیت زمانی باشد، در این صورت حدود ۱۰ درصد زمان پردازنده صرف برگردان متن خواهد شد. کمیت زمانی بیشتر سیستم‌های مدرن، در عمل در بازه‌ی ۱۰ تا ۱۰۰ میلی‌ثانیه تغییر می‌کند. زمان مورد نیاز نوعی برگردان متن کمتر از ۱۰ میکروثانیه می‌باشد؛ بنابراین، زمان برگردان متن فقط کسر کوچکی از کمیت زمانی است.

زمان گردش نیز به اندازه‌ی کمیت زمانی بستگی دارد. همان‌گونه که می‌توانیم از شکل ۵-۶ ببینیم، میانگین زمان گردش مجموعه‌ای از پردازه‌ها، لزوماً با افزایش اندازه‌ی کمیت زمانی بهبود پیدا نمی‌کند. به طور کلی، موقعی در میانگین زمان گردش بهبودی حاصل می‌شود که بیشتر پردازه‌ها انفجار بعدی پردازنده‌ی خود را تنها با یک کمیت زمانی منفرد به اتمام برسانند. برای مثال، اگر سه پردازه که انفجار هر یک از آنها ۱۰ واحد زمان است با کمیت زمانی ۱ واحد داشته باشیم، در این صورت میانگین زمان گردش برابر ۲۹ خواهد بود. با این وجود، اگر کمیت زمانی ۱۰ باشد، میانگین زمان گردش به ۲۰ کاهش می‌یابد. در صورتی‌که زمان برگردان متن نیز به حساب بیاید، چون با کمیت زمانی کوچکتر، برگردان متن بیشتری لازم خواهد شد، میانگین زمان گردش حتی افزایش بیشتری هم خواهد داشت.

اگرچه کمیت زمانی می‌بایست در مقایسه با زمان برگردان متن تا حدی بزرگ انتخاب شود، نباید زیادی هم بزرگ باشد. همان‌گونه که پیشتر نشان دادیم، اگر کمیت زمانی خیلی بزرگ باشد، زمانبندی RR به یک سیاست FCFS تغییر شکل می‌دهد. یک قاعده‌ی سرانگشتی این است که بایستی ۸۰ درصد انفجارهای پردازنده کوتاه‌تر از کمیت زمانی باشند.

۵-۳-۴ زمانبندی الویتی

الگوریتم SJF، حالت خاصی از حالت کلی الگوریتم **زمانبندی الویتی** محسوب می‌شود. برای هر یک از پردازه‌ها، الویتی نسبت داده می‌شود و پردازنده در اختیار پردازه‌ای قرار می‌گیرد که بالاترین الویت را دارد. پردازه‌های با الویت یکسان، به ترتیب FCFS زمانبندی می‌شوند. هر الگوریتم SJF، یک الگوریتم الویتی ساده می‌باشد که الویت (p) آن، عکس انفجار بعدی پردازنده (پیش‌بینی‌شده) است. اگر انفجار پردازنده بزرگ‌تر باشد، الویت آن کمتر است و برعکس.



process	time
P_1	6
P_2	3
P_3	1
P_4	7

شکل ۵-۶ چگونگی تغییرات زمان گردش با کمیت زمانی.

توجه داشته باشید که زمانبندی را با تعابیر الویت *بالا* و الویت *پایین* بحث می‌کنیم. در حالت کلی، الویت‌ها با بازه‌ی ثابتی از اعداد، مانند ۰ تا ۷ یا ۰ تا ۴۰۹۵ تعریف می‌شوند. با این وجود، یک توافق کلی وجود ندارد که ۰ را بالاترین یا پایین‌ترین الویت در نظر بگیریم. برخی سیستم‌ها از اعداد پایین برای نشان دادن الویت کمتر استفاده می‌کنند؛ سیستم‌های دیگری هم هستند که اعداد پایین‌تر را برای الویت بالا به کار می‌برند. این اختلاف نظر می‌تواند موجب سردرگمی شود. در این کتاب، از اعداد پایین برای نشان دادن الویت بالا استفاده می‌کنیم.

به عنوان یک مثال، مجموعه‌ی پردازه‌های زیر را در نظر بگیرید، فرض کنید همگی پردازه‌ها در لحظه‌ی ۰ با ترتیب P_1، P_2، ... P_5 با طول انفجار پردازنده‌ی داده شده برحسب میلی‌ثانیه می‌رسند:

پردازه	زمان انفجار	الویت
P_1	۱۰	۳
P_2	۱	۱
P_3	۲	۴
P_4	۱	۵
P_5	۵	۲

با استفاده از زمانبندی الویتی، پردازه‌ها را مطابق نمودار گانت زیر زمانبندی خواهیم کرد:

P_2	P_5	P_1	P_3	P_4

۰ ۱ ۶ ۱۶ ۱۸ ۱۹

میانگین زمان انتظار، ۸/۲ میلی‌ثانیه است.

الویت‌ها می‌توانند به صورت داخلی و یا خارجی تعریف شوند. الویت‌هایی که به صورت داخلی تعریف می‌شوند برای محاسبه‌ی الویت یک پردازه از یک کمیت یا کمیت‌های قابل اندازه‌گیری استفاده می‌کنند. برای مثال، محدودیت‌های زمانی، نیازهای حافظه، تعداد فایل‌های باز، نسبت میانگین انفجار I/O به میانگین انفجار پردازنده در محاسبه‌ی الویت پردازه‌ها استفاده شده‌اند. الویت‌های خارجی با معیارهایی خارج از سیستم‌عامل انتخاب می‌شوند، نظیر اهمیت پردازه، نوع و مقدار وجهی که در قبال استفاده از کامپیوتر پرداخت می‌شود، دپارتمان حامی کار و سایر موارد، که غالباً عوامل سیاسی هستند.

زمانبندی الویتی می‌تواند پس‌گرفتنی و یا غیرقابل پس‌گیر باشد. زمانی که یک پردازه به صف آماده می‌رسد، الویت آن با الویت پردازه‌ی در حال اجرای فعلی مقایسه می‌شود. اگر الویت پردازه‌ی جدیدالورود از الویت پردازه‌ی در حال اجرا بیشتر باشد، الگوریتم زمانبندی الویتی پس‌گرفتنی، پردازنده را پس می‌گیرد؛ در حالی که، در الگوریتم زمانبندی غیر قابل پس‌گیر، پردازه‌ی جدیدالورود در سر صف آماده قرار داده می‌شود.

مسأله‌ی اصلی الگوریتم‌های زمانبندی الویتی، **انسداد نامعین**[1] یا **قحطی‌زدگی**[2] است. پردازه‌ای که برای اجرا آمادگی دارد، اما به انتظار پردازنده است، مسدود به حساب می‌آید. یک الگوریتم زمانبندی الویتی ممکن است بعضی از پردازه‌های با الویت پایین را به صورت نامعین در حالت انتظار رها کند. در یک سیستم کامپیوتری با بار سنگین که در آن همواره پردازه‌هایی با الویت بالا جریان دارند، ممکن است پردازه‌ای با الویت پایین دیگر نتواند برای همیشه پردازنده را در اختیار بگیرد. به طور کلی، دو حالت می‌تواند اتفاق بیفتد. بالاخره، پردازه اجرا خواهد شد (ساعت ۲ بامداد یکشنبه، زمانی که در نهایت، بار سیستم کاهش می‌یابد) و یا سرانجام با از کار افتادن سیستم کامپیوتری، تمامی پردازه‌های با الویت پایین از بین خواهند رفت (چنان که در Rumor اتفاق افتاد، وقتی کامپیوتر IBM 7094 را در دانشگاه MIT در سال ۱۹۷۳ خاموش کردند، متوجه پردازه‌ای با الویت پایین شدند که در سال ۱۹۶۷ ارسال شده و هنوز هم موفق به اجرا نشده بود.)

یک راه‌حل مسأله‌ی انسداد نامعین پردازه‌های با الویت پایین، **سالخوردگی**[3] است. سالخوردگی، مشمول افزایش تدریجی الویت پردازه‌هایی است که در سیستم به مدت طولانی انتظار کشیده‌اند. برای مثال، اگر بازه‌ی الویت‌ها از ۱۲۷ (پایین) تا ۰ (بالا) باشد، می‌توانیم به‌صورت متناوب (برفرض، هر ثانیه)، الویت هر پردازه‌ی منتظر را ۱ واحد افزایش بدهیم. سرانجام، حتی پردازه‌ای که از ابتدا دارای الویت ۱۲۷ هم باشد، بالاخره به بالاترین الویت رسیده و اجرا می‌شود. در حقیقت، کمی بیشتر از ۲ دقیقه طول نمی‌کشد تا پردازه‌ای با الویت ۱۲۷، پا به سن گذاشته و به پردازه‌ای با الویت ۰ برسد.

گزینه‌ی دیگر این است که زمان‌بندی نوبت–چرخشی و الویتی را طوری ترکیب کنیم که سیستم، پردازه‌ی با الویت بالاترین را اجرا کرده و پردازه‌های با الویت یکسان را با استفاده از

1. Indefinite blocking
2. Starvation
3. Aging

زمان‌بندی نوبت-چرخشی اجرا نماید. اجازه دهید این روال را با یک مثال و استفاده از مجموعه‌ی پردازه‌های زیر که زمان انفجار پردازنده‌ی آن‌ها به میلی‌ثانیه داده شده است نشان دهیم:

پردازه	زمان انفجار	الویت
P_1	۴	۳
P_2	۵	۲
P_3	۸	۲
P_4	۷	۱
P_5	۳	۳

با استفاده از زمان‌بندی الویتی با نوبت-چرخشی برای پردازه‌ها با الویت برابر، ما این پردازه‌ها را مطابق نمودار گانت زیر با استفاده از کمیت زمانی ۲ میلی‌ثانیه زمان‌بندی می‌کنیم:

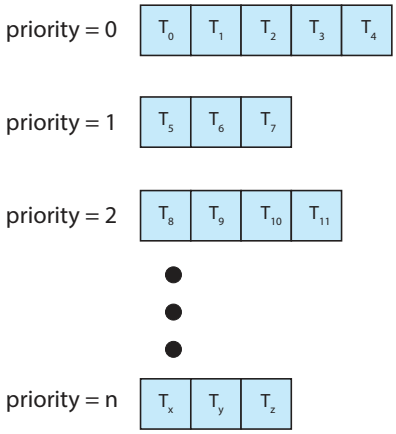
P_4	P_2	P_3	P_2	P_3	P_2	P_3	P_1	P_5	P_1	P_5

۰ ۷ ۹ ۱۱ ۱۳ ۱۵ ۱۶ ۲۰ ۲۲ ۲۴ ۲۶ ۲۷

در این مثال، پردازه‌ی P_4 بالاترین الویت را دارد، لذا تا تکمیل اجرا خواهد شد. پردازه‌های P_2 و P_3 بالاترین الویت بعدی را دارا هستند، و به‌شیوه‌ی نوبت-چرخشی اجرا خواهند شد. توجه کنید که موقعی که پردازه‌ی P_2 در زمان ۱۶ به‌اتمام می‌رسد، پردازه‌ی P_3، پردازه‌ی بالاترین الویت است، لذا اجرا خواهد شد تا این که تکمیل شود. اکنون، تنها پردازه‌های P_1 و P_5 باقی می‌مانند و چون الویت برابری دارند، به‌ترتیب نوبت-چرخشی تا تکمیل شدن اجرا خواهند شد.

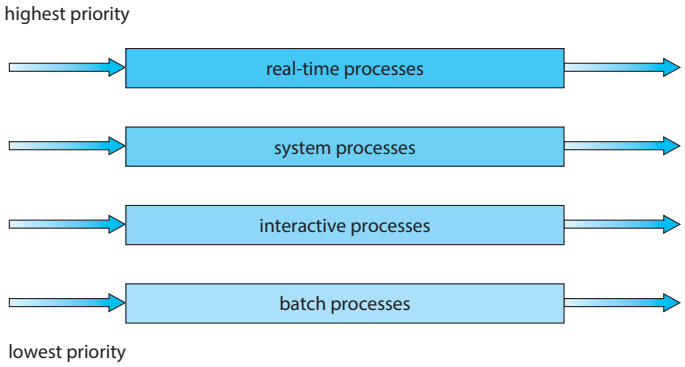
۵-۳-۵ زمانبندی صف چندسطحی

در زمان‌بندی‌های الویتی و نوبت-چرخشی، تمامی پردازه‌ها ممکن است در یک صف منفردی قرار بگیرند و در ادامه، زمان‌بند پردازه‌ی با بالاترین الویت را برای اجرا انتخاب می‌کند. بسته به چگونگی مدیریت صف‌ها، یک جستجوی $O(n)$ ممکن است برای تعیین پردازه‌ی با بالاترین الویت لازم شود. در عمل، غالباً راحت‌تر است که برای هر الویت متمایز، صف‌های جداگانه‌ای داشته باشیم، و زمان‌بندی الویتی به‌سادگی پردازه‌ی صف با بالاترین الویت را زمان‌بندی می‌نماید. این موضوع در شکل ۵-۷ نمایش داده می‌شود. این رویکرد—تحت عنوان **صف چندسطحی**—موقعی که زمان‌بندی الویتی با نوبت-چرخشی ترکیب می‌شود نیز کارگر است: اگر پردازه‌های متعددی در صف با بالاترین الویت وجود داشته باشند، به‌ترتیب نوبت-چرخشی اجرا می‌شوند. در شکل کلی‌تری از این رویکرد، یک الویت به‌صورت ایستا به هر پردازه انتساب می‌شود، و یک پردازه در طی زمان اجرایش در صف یکسانی باقی می‌ماند.



شکل ۵-۷ صف‌های جداگانه برای هر الویت.

یک الگوریتم زمان‌بندی صف چندسطحی می‌تواند برای تفکیک پردازه‌ها به چندین صف مجزا بر اساس نوع پردازه (شکل ۵-۸) نیز استفاده شود. برای مثال، یک تقسیم‌بندی رایج، پردازه‌ها را به پردازه‌های **پیش‌زمینه** (محاوره‌ای) و پردازه‌های **پس‌زمینه** (دسته‌ای) تقسیم می‌کند. این دو نوع پردازه نیازمندی‌های زمان پاسخ متفاوتی دارند و لذا ممکن است نیازهای زمانبندی متفاوتی داشته باشند. به علاوه، ممکن است پردازه‌های پیش‌زمینه نسبت به پردازه‌های پس‌زمینه الویت (با تعریف خارجی) بگیرند. صف‌های مجزایی ممکن است برای پردازه‌های پیش‌زمینه و پس‌زمینه استفاده شوند، و هر صف امکان دارد که الگوریتم زمان‌بندی خاص خودش را داشته باشد. برای مثال، صف پیش‌زمینه می‌تواند توسط یک الگوریتم RR زمانبندی شود، در حالی‌که صف پس‌زمینه با الگوریتم FCFS زمانبندی می‌شود.



شکل ۵-۸ زمان‌بندی صف چندسطحی.

به‌علاوه، بایستی مابین صف‌ها نیز زمانبندی صورت گیرد که در بیشتر موارد به صورت زمانبندی پس‌گرفتنی با الویت ثابت پیاده‌سازی می‌شود. برای مثال، صف بلادرنگ می‌تواند نسبت به صف محاوره‌ای الویت مطلق داشته باشد.

بیایید یک مثال از الگوریتم زمانبندی صف چندسطحی را با چهار صف که در زیر به ترتیب الویت لیست شده‌اند ببینیم:

۱- پردازه‌های بلادرنگ
۲- پردازه‌های سیستمی
۳- پردازه‌های محاوره‌ای
۴- پردازه‌های دسته‌ای

هر صف نسبت به صف‌های با الویت پایین‌تر، الویت مطلق دارد. برای مثال، پردازه‌ای از صف دسته‌ای نمی‌تواند اجرا شود مگر این که، صف‌های پردازه‌های بلادرنگ، سیستمی، و محاوره‌ای همگی خالی شده باشند. اگر یک پردازه‌ی محاوره‌ای به صف آماده وارد شود که در همان حال، یک پردازه‌ی دسته‌ای اجرا دارد، پردازه‌ی دسته‌ای پس‌گرفته خواهد شد.

حالت دیگری که امکان‌پذیر است، اعمال سهم زمانی مابین صف‌هاست. در اینجا، هر صف، سهم معینی از زمان پردازنده را در اختیار می‌گیرد که می‌تواند آن زمان را مابین پردازه‌های مختلف خود زمانبندی کند. برای نمونه، در مثال صف پیش‌زمینه–پس‌زمینه، می‌توان ۸۰ درصد زمان پردازنده را برای صف پیش‌زمینه در نظر گرفت تا مابین پردازه‌های خود از زمانبندی RR استفاده کند و ۲۰ درصد زمان پردازنده را در اختیار صف پس‌زمینه قرار داد تا آن را مطابق FCFS به پردازه‌های خود تخصیص بدهد.

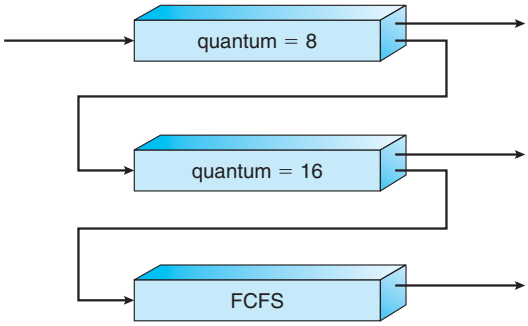
۵-۳-۶ زمانبندی صف با بازخورد چندسطحی

زمانی که از الگوریتم زمانبندی صف چندسطحی استفاده می‌شود، پردازه‌ها در اصل در زمان ورود به سیستم به طور دایمی تنها به یک صف گمارده می‌شوند. برای مثال، اگر برای پردازه‌های پیش‌زمینه و پس‌زمینه، صف مجزایی وجود داشته باشد، پردازه‌ها نمی‌توانند از یک صف به صف دیگر جابه‌جا شوند، چون پردازه‌ها نمی‌توانند طبیعت پیش‌زمینه یا پس‌زمینه‌بودن خود را تغییر بدهند. مزیتی که برپایی این مدل صف دارد پایین بودن سربار زمانبندی است، اما انعطاف‌پذیر نیست.

در مقایسه، یک پردازه در **الگوریتم زمانبندی صف با بازخورد چندسطحی**[1] می‌تواند میان صف‌ها جابه‌جا شود. اصل آن است که پردازه‌ها مطابق مشخصات انفجار پردازنده‌ای که دارند از هم تفکیک شوند. اگر پردازه‌ای برای زمان بیشتری از پردازنده استفاده می‌کند، به یک صف با الویت پایین‌تر جابه‌جا خواهد شد. این طرح، پردازه‌های پاپیچ I/O و محاوره‌ای— که نوعاً با انفجارهای کوتاه پردازنده مشخص می‌شوند—را در صف‌های با الویت بالاتر قرار می‌دهد. به علاوه، پردازه‌ای که در یک صف با الویت پایین‌تر برای مدت زمان طولانی منتظر می‌ماند می‌تواند به یک صف با الویت بالاتر جابه‌جا شود. این شکل سالخوردگی نیز از قحطی‌زدگی جلوگیری می‌کند.

1. Multilevel feedback-queue

برای مثال، یک زمانبند صف با بازخورد چندسطحی را با سه صف که از ۰ تا ۲ شماره‌گذاری شده‌اند، در نظر بگیرید (شکل ۵-۹). زمانبند در ابتدا تمامی پردازه‌های صف ۰ را اجرا می‌کند. تنها زمانی که صف ۰ خالی شود پردازه‌های صف ۱ را اجرا خواهد کرد. به طور مشابه، تنها در صورتی پردازه‌های صف ۲ اجرا می‌شوند که صف‌های ۰ و ۱ خالی شده باشند. پردازه‌ای که به صف ۱ می‌رسد پردازه‌ی صف ۲ را پس خواهد گرفت. پردازه‌ی صف ۱ نیز به نوبت خود توسط پردازه‌ای که به صف ۰ می‌رسد پس‌گرفته می‌شود.



شکل ۵-۹ صف‌های با بازخورد چندسطحی.

پردازه‌ی ورودی در صف ۰ قرار داده می‌شود. پردازه در صف ۰، یک کمیت زمانی ۸ میلی‌ثانیه به‌دست می‌آورد. اگر در این زمان پایان نپذیرد، به ته صف ۱ جابه‌جا می‌شود. اگر صف ۰ خالی شود، پردازه‌ای که در سر صف ۱ قرار دارد، یک کمیت ۱۶ میلی‌ثانیه به‌دست می‌آورد. اگر باز هم تکمیل نشود، پس‌گرفته شده و در ته صف ۲ قرار می‌گیرد. پردازه‌های صف ۲ مطابق FCFS، فقط زمانی که صف‌های ۰ و ۱ خالی شده باشند اجرا خواهند داشت. برای پیشگیری از قحطی‌زدگی، پردازه‌ای که به مدت طولانی در یک صف با الویت پایین‌تر منتظر مانده است ممکن است تدریجاً به یک صف با الویت بالاتر انتقال داده شود.

این الگوریتم زمانبندی، بالاترین الویت را به پردازه‌ای می‌دهد که انفجار پردازنده‌اش برابر ۸ میلی‌ثانیه یا کمتر است. چنین پردازه‌ای پردازنده را به سرعت در اختیار گرفته، انفجار پردازنده‌اش را به اتمام می‌رساند و سراغ انفجار I/O بعدی خود می‌رود. پردازه‌هایی که به بیشتر از ۸ میلی‌ثانیه، اما کمتر از ۲۴ میلی‌ثانیه نیاز دارند، هرچند با الویت پایین‌تری از پردازه‌های کوتاه‌تر، باز هم زودتر سرویس می‌گیرند. پردازه‌های طولانی به طور خودکار به صف ۲ روانه شده و به ترتیب FCFS با دورهای باقیمانده‌ی پردازنده از صف‌های ۰ و ۱ سرویس می‌گیرند.

در حالت کلی، یک زمانبند صف با بازخورد چندسطحی توسط پارامترهای زیر تعریف می‌شود:

- تعداد صف‌ها
- الگوریتم زمانبندی هر یک از صف‌ها
- روشی که برای تعیین زمان ارتقای یک پردازه به یک صف با الویت بالاتر استفاده می‌شود
- روشی که برای تعیین زمان تنزل یک پردازه به یک صف با الویت پایین‌تر استفاده می‌شود